#show table.cell: set par(justify: false) #show table.cell: set text(size: 9pt, hyphenate: false)

ФСП Таланты

Платформа подбора ИТ-специалистов с подтверждённой категорией. Сопроводительная документация

Содержание

ФСП Таланты — платформа подбора ИТ-специалистов с подтверждённой категорией 2
Коротко о решении 2
1. Функциональная архитектура 3
2. Компонентная архитектура 5
3. Справочники 6
4. Механика тестирования 6
5. Механика подбора и категоризация 8
6. Процедура валидации и результаты 10
7. Интеграция с ФСП 12
8. Безопасность и 152-ФЗ 13
9. API 15
10. Антифейк и концепции вне MVP 16
11. Библиотеки и компоненты 23
12. Сборка, развёртывание и запуск 24

ФСП Таланты — платформа подбора ИТ-специалистов с подтверждённой категорией

Хакатон «Лидеры цифровой трансформации 2026», спецтрек Федерации спортивного программирования России.

Репозиторий: https://github.com/K1NGSM4N3/fsp-hack · Swagger UI развёрнутого решения: `/api/docs` · Отчёты валидации: `docs/validation/`

Коротко о решении

Работодатель не ждёт откликов, а сам выходит на кандидата. Кандидаты разложены по **категориям** — специализация × грейд, — и категорию даёт только **адаптивный тест**, а не резюме. Внутри категории выше стоят те, у кого сильнее результат теста, подтверждены навыки и есть достижения ФСП. Работодатель описывает потребность, получает **объяснённую подборку** и отправляет **приглашение с обязательной зарплатной вилкой**. Кандидат видит условия до начала общения, а его контакты открываются работодателю только после согласия.

Что отличает решение:

1. **Тест, который бесполезно сливать.** Задания — параметрические шаблоны: у каждого кандидата свои числа, код и ответы, а ответ вычисляется генератором. Сложность сопоставима, потому что шаблоны откалиброваны по IRT и все кандидаты измеряются на одной шкале. В симуляции утечки у фиксированного теста при 50 % слитых ответов проходят 99,9 % слабых кандидатов, у нашего — 9–10 %, как и без утечки (§6.2).
2. **Механика контакта «работодатель ⟶ кандидат».** Анонимная карточка, объяснение попадания, вилка в первом сообщении, квота приглашений против спама, причины отказов обратно работодателю, статистика спроса кандидату (§5).
3. **Измеримое качество.** Собственная процедура валидации на синтетике со скрытой «истиной»: доля релевантных в топ-10 — 0,94 против 0,43 у поиска по ключевым словам резюме; итоговый грейд совпадает с истинным в 87 % случаев, при пересдаче — $\kappa = 0{,}74$ (§6). Свой набор вакансий, резюме и разметки можно прогнать через рабочий подбор одной командой или из Swagger (§6.4).
4. **ФСП ID на Keycloak уже сейчас.** Привязка через OIDC + PKCE к заглушке ФСП ID на настоящем Keycloak 26, достижения из реестра влияют на ранжирование, кандидат без истории ФСП не штрафуется (§7).

5. **Видеозапись теста — по согласию и без потери анонимности.** Камера предлагается перед тестом, запись шифруется и хранится 90 дней; кандидат видит свою запись, работодатель — только отметку «Тест с видеозаписью» и число уходов со вкладки (§4.7, §8.2).

1. Функциональная архитектура

1.1. Роли и сквозные сценарии

Роль	Сценарий
Кандидат	Регистрация по email с подтверждением и согласиями (с 14 лет; до 18 — с согласием законного представителя по ссылке из письма) ⟶ профиль (форма или PDF-резюме с распознаванием) ⟶ опрос (направление, специализация, стек, заявленный грейд) ⟶ адаптивный тест ⟶ категория ⟶ входящие приглашения с вилкой ⟶ принять / отклонить с причиной. Дополнительно: вакансии и отклики, регулярные задания, привязка ФСП ID, PDF-профиль, настройки приватности и согласий, журнал «Кто открывал ваши контакты», выгрузка своих данных
Работодатель	Профиль компании ⟶ описание потребности (свободным текстом, система распознаёт специализацию, грейд, стек и вилку) ⟶ рекомендованные категории ⟶ подборка с объяснениями, уточнение фильтров без потери ⟶ анонимный профиль ⟶ приглашение с вилкой ⟶ контакты после принятия. Дополнительно: поиск по всему банку, вакансии и отклики, регулярные задания, статистика приглашений, раздел «Доверие и квота», подключение своей ATS (вебхуки и API-ключи)

1.2. Покрытие требований ТЗ

Требование	Реализация
Категория = специализация × грейд по опросу и тесту, не по резюме	6 специализаций × 3 грейда; категорию присваивает только сданный тест (§4, §5.3)
Механика тестирования, устойчивая к утечкам, с сопоставимой сложностью	62 параметрических шаблона, IRT 3PL, адаптивный тест (§4)
Грейд не понижается принудительно; пересдача ниже / выше; ограничение по времени	Политика грейда: шаг вниз сразу после провала, дотест вверх после уверенной сдачи, смена грейда не чаще раза в 90 дней (§4.5)
Ранжирование внутри категории по тесту и ФСП	Сила профиля = 0,6 · тест + 0,25 · ФСП + 0,15 · актуальность (§5.3)
Работодатель находит категорию и выходит на кандидата с предложением и зарплатой	Рекомендованные категории, подборка, поиск по банку, приглашение без привязки к вакансии (§5)
Объяснимость выдачи; уточнение без потери подборки	Разложение балла по признакам, причины и пробелы; подборка хранится, закреплённые кандидаты переживают уточнение (§5.4)
Зарплата «от–до» обязательна; контакты после принятия / отклика	Проверяется в API и ограничениях БД; контакты недоступны на уровне типов (§8.3)
Статусы приглашений и откликов	отправлено ⟶ просмотрено ⟶ принято / отклонено; истекло, отозвано
152-ФЗ: согласия на обработку и публикацию, права по ролям	Версионированные согласия; профиль публикуется только при действующем согласии; соискатели 14–17 лет — только с согласием законного представителя; политика обработки `/privacy`; выгрузка своих данных и журнал доступа к контактам в кабинете (§8.2)
Связь с ФСП ID; корректная обработка без истории ФСП	OIDC + PKCE к Keycloak, реестр достижений, нейтральное «Нет данных ФСП» (§7)
PDF-резюме с распознаванием, стандартизированный PDF-профиль	Извлечение текста + словари и правила; PDF в фирменном шрифте ФСП
Регулярные задания, вакансии и отклики (желательно)	Реализованы
Защита от фиктивных вакансий (концепция, рабочая часть — плюс)	Работает: подтверждение корпоративного домена, сверка вилки с рынком категории, поиск почти одинаковых вакансий, уровень доверия компании, от которого зависит квота приглашений, очередь модерации (§10.1). Дальше — ЕГРЮЛ и модерация с решениями
Интеграция с ATS работодателей (концепция)	Реализован универсальный слой: подписанные вебхуки из транзакционного outbox, API-ключи со скоупами, этапы найма от ATS, импорт вакансий, демо-ATS; план нативных коннекторов (§10.2)

1.3. Интерфейс

Публичный сайт — пять страниц, у каждой свой адресат: главная (`/`) показывает механику на примере одного приглашения; `/seekers` — путь соискателя от теста до приглашения с вилкой и мини-демо адаптивного теста; `/business` — как компания описывает потребность и получает подборку, правила для всех компаний; `/fsp` — что даёт привязка ФСП ID и как разряды и призовые места попадают в профиль; `/vacancies` — опубликованные вакансии с фильтрами. Отдельно — политика обработки персональных данных `/privacy` с действующими текстами согласий и страница законного представителя `/guardian/confirm`. Страницы рендерятся на сервере и читаются

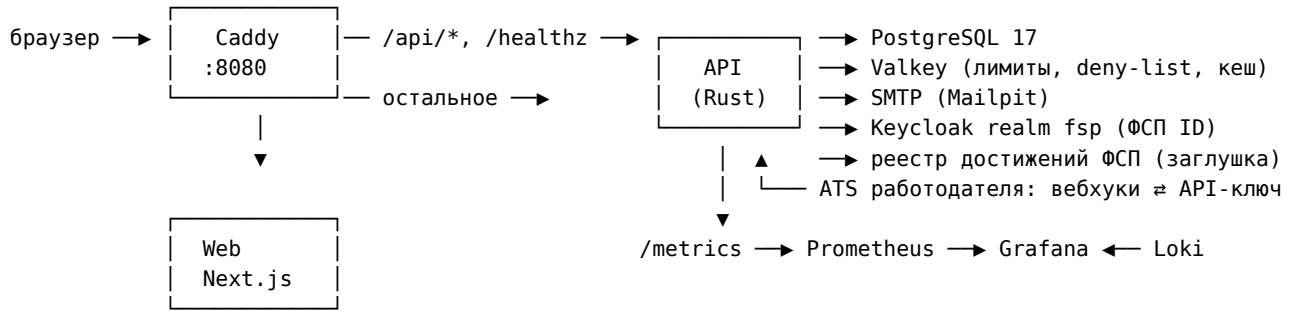
без JavaScript; анимации загружаются после первой отрисовки, на телефонах облегчены, при `prefers-reduced-motion` отключены (§2).

Кабинет кандидата: главная (категория, приглашения, спрос в категории, ФСП), профиль и резюме, опрос, тестирование с шагом видеозаписи, приглашения, вакансии и отклики, регулярные задания, настройки (статус поиска, видимость, ФСП ID, согласия, «Кто открывал ваши контакты», выгрузка данных, удаление аккаунта).

Кабинет работодателя: главная (квота, воронка приглашений, причины отказов), потребности и подборки, поиск по базе, приглашения, вакансии и отклики, регулярные задания, интеграции, компания, доверие и квота.

Вёрстка адаптивная от 320 px; кабинеты на телефоне работают с выдвижным меню.

2. Компонентная архитектура



Backend — модульный монолит на Rust. Логика вынесена в чистые крейты без ввода-вывода и часов, поэтому одна и та же реализация работает в API и в симуляторе валидации:

Крейт	Назначение
`catalog`	Справочники: направления, специализации, грейды и их пороги на шкале θ, категории, отрасли
`assessment`	Тестирование: модель IRT, адаптивный выбор заданий, 62 шаблона, политика смены грейда
`matching`	Подбор: словарь навыков с синонимами и графом близости, скоринг с объяснениями, ФСП-скор, разбор текста вакансии и резюме
`db`	Доступ к PostgreSQL на sqlx: SQL-запросы, проверенные по схеме при компиляции, миграции, тесты на живой базе
`api`	HTTP API (axum), аутентификация, OpenAPI, сервисы
`fsp-mock`	Заглушка реестра достижений ФСП
`ats-mock`	Демо-ATS работодателя: принимает подписанные вебхуки, отправляет этапы найма обратно (§10.2)
`validate`	Процедура валидации, формирует `docs/validation/`

Frontend — Next.js 16 (App Router, TypeScript). Публичные страницы рендерятся на сервере, кабинеты — клиентские, данные через TanStack Query; типы API генерируются из OpenAPI. Права проверяет только API. Анимации публичного сайта (GSAP, Lenis) приходят отдельным чанком после гидратации и работают в двух профилях: на десктопе — пины, прокрутка со scrub, разбивка заголовков, плавный скролл; на телефонах и планшетах — одноразовые появления только через transform и opacity и нативный скролл.

Данные: PostgreSQL — пользователи (email и ФИО зашифрованы), профили, навыки с источником (заявлен / из резюме / подтверждён тестом), сессии тестов (хранятся только ссылки на экземпляры заданий: шаблон, уровень, seed), история грейдов, потребности и сохранённые подборки, приглашения и их история, журнал доступа к контактам, согласия и запросы законным представителям, метаданные видеозаписей (сами фрагменты — зашифрованными файлами на диске), очередь событий для ATS. Схема — 10 миграций sqlx, применяются при старте API.

Развёртывание: docker compose с профилями — ядро (Caddy, web, API, PostgreSQL, Valkey, Mailpit), `fsp` (Keycloak и реестр), `ats` (демо-ATS), `obs` (Prometheus, Loki, Alloy, Grafana), `full` — всё. Лимиты памяти контейнеров профиля `full` в сумме — 3 200 МиБ (из них Keycloak — 768, PostgreSQL — 512, API — 384); весь стек укладывается в недорогой VPS.

3. Справочники

- **Направления и специализации:** разработка (бэкенд, фронтенд), инфраструктура (DevOps/SRE), качество (QA), данные и ИИ (ML/DS), безопасность.
- **Грейды:** Junior, Middle, Senior. Пороги на шкале способности θ: −1, 0, +1.
- **Навыки:** 83 навыка, 491 синоним на русском и английском, включая сленг («постгрес», «кубер», «джава»), 77 рёбер близости (PostgreSQL ~ MySQL, React ~ Vue…).

4. Механика тестирования

4.1. Идея

Единый набор заданий утекает; генерация нейросетью «под каждого» не гарантирует сопоставимую сложность. Мы разделили две задачи:

- **Неповторимость** даёт генератор: шаблон задания по случайному `seed` выдаёт новый экземпляр — другие числа, код, имена, порядок и набор вариантов. Правильный ответ **вычисляется** генератором (мини-симулятор event loop JavaScript, расчёт подсети, выполнение SQL на сгенерированной таблице и т. п.), а не хранится.
- **Сопоставимость** даёт калибровка: у каждого уровня шаблона есть параметры модели IRT — дискриминативность `a`, сложность `b`, угадывание `c`. Способность всех кандидатов измеряется на общей шкале θ, поэтому разные задания дают сравнимый результат.

Сложность внутри шаблона задают «радикалы» (например, число асинхронных задач в event loop), а «инциденталы» (числа, имена, порядок) меняются без влияния на сложность.

4.2. Банк заданий

62 шаблона: 52 вычислительных генератора и 10 вариативных пулов (5–7 формулировок, 3–4 равноправных правильных ответа и 7–10 дистракторов на каждый вопрос). У каждого шаблона 1–3 уровня сложности, $b \approx -1{,}4 \ldots +1{,}4$. Модульный тест закрепляет разнообразие: на 500 случайных `seed` каждый уровень каждого шаблона даёт не меньше 490 различимых экземпляров — при N равновероятных вариантах это значит, что N исчисляется десятками тысяч.

Специализация	Шаблонов	Тем	Примеры
Бэкенд	24	10	изменяемый аргумент по умолчанию в Python, порядок `defer` в Go, выбор индекса, token bucket, LRU, аномалии изоляции, SQL `GROUP BY / HAVING`
Фронтенд	15	6	порядок вывода в event loop, замыкания в цикле, приведение типов, специфичность CSS, box model, батчинг состояния React
DevOps	18	10	кэш слоёв Docker, ресурсы Kubernetes, CIDR, `location` в nginx, cron, права Linux, `increase()` в Prometheus, бюджет ошибок
QA	18	8	граничные значения, классы эквивалентности, pairwise, диаграммы состояний, таблицы решений, проверки API
ML	18	8	метрики по матрице ошибок, теорема Байеса, градиентный шаг, k-means, `groupby` в pandas, формы массивов NumPy
Безопасность	15	8	класс уязвимости во фрагменте кода, энтропия пароля, XOR-шифр, Base64, обход фильтров, правила межсетевого экрана

Типы ответа: выбор одного, выбор нескольких, число, текст. Задания и интерфейс — на русском, код — на соответствующем языке.

4.3. Адаптивный тест

Параметр	Значение
Модель	3PL; $c = 1/k$ для выбора одного из k вариантов, 1/31 для «выберите все верные» из пяти, 0 для числового и текстового ответа
Оценка θ	Апостериорное среднее (EAP), сетка из 81 точки на [−4; 4], априорное N(порог заявленного грейда; 1)
Выбор задания	Максимум информации Фишера в текущей θ с поправкой на баланс тем (темы с недобором получают приоритет), случайный выбор среди пяти лучших (контроль экспозиции), без повторов шаблона
Остановка	Не менее 8 и не более 20 заданий; стоп, когда 95-процентный интервал по θ не содержит порог заявленного грейда
Решение	$\theta \geqslant$ порога $\longrightarrow$ тест сдан
«Уверенно сдал»	Измеренный грейд выше заявленного или $P(\theta \geqslant$ следующего порога$) \geqslant 0{,}9 \longrightarrow$ предлагается дотест на грейд выше
Время	Лимит на задание (60–180 с), просроченный ответ засчитывается как неверный
Аномалии	Статистика person-fit l_z («верно на трудных, неверно на лёгких», $l_z < -2$); вместе с событиями видеозаписи (уход со вкладки, вставка текста) результат помечается «требует подтверждения» — §4.7

Стек кандидата из опроса повышает приоритет шаблонов с совпадающими навыками (× 1,5), но шкала и пороги остаются общими. Если тест сдан, навыки верно решённых заданий получают отметку «подтверждён тестом».

4.4. Почему тест устойчив к распространению заданий

Что утекло	Что даёт утечка
Конкретный экземпляр с ответом	Практически ничего: следующий кандидат получит другой seed, а значит другой код и другой ответ
Шаблон («тут спрашивают порядок вывода event loop»)	Понимание, как решать такой класс задач, — ровно тот навык, который мы измеряем
Весь банк шаблонов	Ответы всё равно нужно вычислить, а на каждое задание действует лимит времени

Количественная проверка — §6.2: даже при базе из ~44 тыс. слитых экземпляров совпадения встречаются в 0,02–0,05 % заданий, и доля прошедших слабых кандидатов не растёт (9–10 %), тогда как у фиксированного теста — до 100 %.

Роль нейросетей: только офлайн — предложить новый шаблон, который проходит ревью и пробную калибровку (показывается кандидатам без влияния на оценку) до попадания в рабочий банк. Сопоставимость обеспечивает калибровка, а не генерация.

4.5. Правила смены грейда

- **Первичное размещение.** Кандидат выбирает грейд и проходит тест. Не сдал — сразу может пройти тест на грейд ниже; повтор того же или более высокого грейда — через 30 дней (иначе тест можно было бы перебирать). Не сдал Junior — пересдача через 30 дней, категории пока нет. Брошенный тест через 30 минут после дедлайна текущего задания закрывается как истёкший и в этих правилах не учитывается.
- **Уверенно сдал** — один раз сразу предлагается дотест на грейд выше, в том числе после уверенной сдачи на шаг ниже.
- **После присвоения** смена грейда вверх или вниз — не чаще раза в 90 дней (настраивается). Неудачная попытка повышения не понижает грейд; следующая — через 30 дней, но не раньше, чем закончится 90-дневное окно от присвоения.

- Кандидат видит текущую категорию, историю опросов, тестов и присвоений, доступные попытки и дату, когда откроется смена грейда.

4.6. Как тест строится от вакансии

`GET /api/v1/catalog/test-plan` и `GET /api/v1/employer/needs/{id}/test-plan` показывают по описанию вакансии: целевой порог θ, темы, шаблоны, которые получат приоритет из-за стека вакансии, и примеры заданий для двух разных кандидатов. В демо-данных у работодателя `employer@demo.fsp` шесть потребностей — по одной на специализацию: Go-разработчик в платежи, фронтенд личного кабинета мерчанта, DevOps платформы процессинга, QA по автоматизации API выплат, ML-инженер антифрода и AppSec-инженер платёжного API. У каждой свой план теста («Как проверяется категория») и непустая подборка. Например, для «Middle Go-разработчик: PostgreSQL, Kafka» приоритет получают `sql.group_by_having`, `sql.left_join_rows`, `go.defer_order`, `db.index_choice`, `db.isolation_anomaly`. Кандидаты А и Б получают одни и те же шаблоны и уровни, но разные запросы и данные — `WHERE o.amount > 600` у одного и `> 750` у другого.

4.7. Видеозапись теста и журнал событий

Это наше дополнение: ТЗ прокторинга не требует. Цель — подтвердить, что тест пройден самостоятельно, не нарушив приватность и не мешая отвечать. Режим задаётся переменной `PROCTORING=optional|off`; по умолчанию камера предлагается, но не обязательна.

Перед тестом кандидат видит шаг «Видеозапись во время теста»: предпросмотр камеры и три коротких пункта — что записываем (только видео с камеры, без звука и без экрана, в низком качестве), кто увидит (сам кандидат и администратор при разборе спора; работодатель — только отметку) и сколько храним (90 дней в зашифрованном виде). Галочка согласия — отдельный версионированный документ в общей системе согласий. Две кнопки: «Начать с видеозаписью» и «Продолжить без камеры». Без камеры тест засчитывается так же, у результата отметка «без видеозаписи».

Во время теста браузер записывает видео 320×240 с низким битрейтом и каждые 10 секунд отправляет фрагмент на сервер; в углу — индикатор «Идёт запись» с миниатюрой камеры. Отдельно, с камерой и без, отмечаются события: вкладка скрыта, окно потеряло фокус, выход из полноэкранного режима (его предлагаем, но не навязываем), вставка текста в поле ответа, камера отключилась, запись остановилась. Отправка фоновая и с повторами; любой сбой превращается в «запись прервалась», тест при этом продолжается.

Итог — сводка с одним из статусов «с видеозаписью», «без видеозаписи», «запись прервалась» и числом событий каждого типа. Она пересчитывается из сохранённых данных при каждом чтении, поэтому учитывает последний фрагмент, который рекордер отдаёт уже после последнего ответа (сервер принимает его ещё 60 секунд).

«Требует подтверждения». Ни флаг person-fit, ни уход со вкладки по отдельности не доказывают помощь со стороны: порог $l_z < -2$ срабатывает примерно у 2 % честных кандидатов, а переключить вкладку может любой. Поэтому отметка ставится, только когда аномальный шаблон ответов совпал с уходами со вкладки, вставкой текста или выходом из полноэкранного режима. Грейд автоматически не меняется: отметку видят кандидат и администратор, который может посмотреть запись.

Кто	Что видит
Кандидат	Сводку в результате и в истории тестов, свою запись, дату её удаления и ровно ту отметку, которую видят компании
Работодатель	Только отметку в анонимной карточке и профиле: «Тест с видеозаписью», «Тест без видеозаписи», «Запись прервалась», «Есть замечания: 2 ухода со вкладки». Видео недоступно ни до, ни после принятия приглашения
Администратор	Журнал событий и запись любой сессии — для разбора спорного результата

5. Механика подбора и категоризация

5.1. Категоризация

Категория присваивается только по результату теста: сданный тест на заявленный грейд ⟶ категория «специализация · грейд». Самоописание и резюме влияют лишь на вторичные признаки (стек, ожидания) и никогда не определяют категорию. Кандидат без категории в поиске не участвует.

5.2. Сценарий выхода на контакт

1. Работодатель описывает потребность — формой или вставкой текста вакансии; система распознаёт специализацию, грейд, обязательные и желательные навыки, вилку, формат.
2. Система показывает **рекомендованные категории**: сколько кандидатов, сколько с ФСП, медианные ожидания, насколько категория покрывает стек.
3. **Подборка** — анонимные карточки: категория, сила результата теста, навыки с отметкой «подтверждён тестом / из резюме / заявлен», достижения ФСП, опыт (названия компаний — по желанию кандидата), ожидания. Без имени и контактов.
4. **Объяснение** у каждого кандидата: балл 0–100, разложение по признакам, причины («Все обязательные навыки: Go, PostgreSQL, Kafka; 1 подтверждён тестом», «Результат теста выше 71 % диапазона грейда Middle», «ФСП: Мастер спорта по продуктовому программированию, 2023») и пробелы («Нет обязательного навыка: Kubernetes», «Вместо PostgreSQL — близкий навык MySQL», «Навык Docker только заявлен»).
5. **Приглашение**: что предлагаем, сообщение, вилка «от–до» (обязательно), формат, способ связи с компанией; можно без вакансии.
6. Кандидат видит условия до общения и принимает или отклоняет с причиной (зарплата, стек, формат, компания, не ищу работу).
7. После принятия работодатель получает контакты; каждое обращение к контактам записывается в журнал.

5.3. Ранжирование

Жёсткие фильтры: кандидат видим и открыт к предложениям, категория нужной специализации, грейд из запрошенных (или соседний, если разрешено), формат и город совместимы.

Балл — взвешенная сумма признаков из [0; 1], веса подобраны на валидации (§6.3):

Признак	Вес	Расчёт
Обязательные навыки	0,41	Среднее по навыкам потребности: близость × доверие к источнику (тест — 1,0; резюме — 0,7; заявлен — 0,5)
Грейд	0,15	1,0 — точное совпадение, 0,6 — на грейд выше, 0,5 — на грейд ниже
Результат теста	0,10	Положение θ внутри диапазона грейда
Достижения ФСП	0,10	ФСП-скор (§7.3)
Желательные навыки	0,08	Как обязательные
Зарплата	0,07	1,0, если ожидания в вилке; линейно до 0 при превышении на 30 %; ожидания не указаны — нейтральные 0,7
Актуальность	0,05	exp(−дней с последней активности / 60): тест, задание, правка профиля, опрос, синхронизация ФСП, отклик, ответ на приглашение
Близость текстов	0,04	Полнотекстовая релевантность «о себе» описанию потребности (русская морфология); если в потребности указаны отрасли — 0,6 · текст + 0,4 · совпадение отраслей опыта

Признаки без данных (нет желательных навыков, нет описания и отраслей) выпадают, веса перенормируются. Без потребности — при просмотре категории — кандидаты упорядочены по силе профиля: 0,6 · тест + 0,25 · ФСП + 0,15 · актуальность.

Подбор 10 000 кандидатов с объяснением топ-20 занимает около 0,3 мс; в ранжирующем цикле нет аллокаций, тексты объяснений собираются только для итоговых кандидатов. Пул специализации API держит в памяти и сбрасывает при каждом изменении, влияющем на выдачу: при 10 000 кандидатах и 50 одновременных клиентах поиск по банку под потребность отвечает за 43 мс (p95) — §12.4.

5.4. Уточнение без потери подборки

Подборка хранится. При уточнении фильтров она пересчитывается, но закреплённые кандидаты остаются (с пометкой, если не проходят новые фильтры), скрытые не возвращаются, а новые помечаются как «новые».

5.5. Чем это отличается от отклика на вакансию

- Инициатива у работодателя, который ищет по подтверждённым тестом категориям, а не по ключевым словам.
- Условия — в первом же сообщении: кандидат решает, тратить ли время.
- **Квота**: до 30 приглашений компании за 7 дней — сколько именно, зависит от уровня доверия (§10.1): новая компания начинает с
  10. Плюс одно действующее приглашение на пару «компания–кандидат».

  Работодатель выбирает точечно, кандидат не тонет в спаме.
- Приглашение живёт 14 дней. Работодатель видит воронку и агрегированные причины отказов, кандидат — анонимную статистику спроса в своей категории: сколько компаний смотрели профиль, медианная вилка приглашений.
- Классический отклик остаётся дополнительным путём: контакты раскрываются и при собственном отклике.

6. Процедура валидации и результаты

Валидация — отдельный исполняемый модуль `crates/validate` (`make validate`), который вызывает **те же крейты**, что и API, и пишет отчёты в `docs/validation/`. Данных и разметки организаторы не дают, поэтому используется синтетика со скрытой «истиной».

6.1. Тестирование: точность и стабильность

Каждая сессия — настоящий движок; синтетические только ответы: вероятность верного ответа по 3PL при истинной $\theta \sim N(0; 1)$. 400 человек на специализацию, процедура как в API (заявленный грейд → при провале грейд ниже). Пересдача — вторая независимая процедура с другими заданиями.

Сценарий	Решение «выше/ниже порога» верно	Итоговый грейд = истинный	Отклонение ⩽ 1 грейда	Совпадение при пересдаче	κ Коэна	RMSE θ	Заданий
Честная заявка, точная калибровка	87,7 %	87,0 %	100 %	81,1 %	0,74	0,35	13,6
20 % завышают, 10 % занижают грейд	87,7 %	80,2 %	99,2 %	80,1 %	0,72	0,35	13,4
Ошибка калибровки заданий (a ×0,8…1,2, b ±0,3)	87,1 %	86,2 %	100 %	80,2 %	0,72	0,35	13,6

Ошибки только «вниз» и только у границы грейда: тест на заявленный грейд не повышает принудительно, а пограничный кандидат может пересдать на грейд выше. Неточная калибровка почти не ухудшает результат.

Дискриминативность: точечно-бисериальная корреляция верного ответа с истинной θ — в среднем 0,30, у 94 % шаблонов $\geqslant 0{,}2$. Самые слабые шаблоны выводятся в отчёт как кандидаты на перекалибровку. Низкая корреляция у шаблона с несколькими уровнями может быть артефактом смешения уровней: адаптивный тест показывает трудный уровень более сильным кандидатам, поэтому внутри уровня корреляция выше, чем по шаблону в целом (так было с `go.defer_order`: 0,30 / 0,21 / 0,50 по уровням при 0,13 в сумме; уровень перекалиброван).

6.2. Тестирование: устойчивость к утечке

Слабые кандидаты (истинная θ от −0,9 до −0,1, ниже порога Middle) сдают тест на Middle.

Утекло	Фиксированный тест: прошли	Наш тест: прошли
0 %	9,4 %	9,7 %
25 % ответов	65,2 %	9,6 %
50 % ответов	99,9 %	9,3 %
100 % ответов	100 %	10,4 %

Фиксированный тест — публичная форма из 15 самых информативных заданий. Для нашего теста «утечка» — база из всех экземпляров, показанных в 600 предыдущих сессиях на специализацию (44 306 экземпляров); слабый

кандидат отвечает верно, если ему выпал экземпляр из базы. Совпадения с базой случаются в 0,02–0,05 % заданий, поэтому утечка не даёт преимущества. Статистика person-fit ловит 11–12 % «сливщиков» фиксированного теста при 0,9 % ложных срабатываний.

6.3. Подбор: доля релевантных в топе

2 400 синтетических кандидатов, 120 потребностей для подбора весов и 120 отложенных для оценки. У кандидата есть скрытая истина (реальные навыки и θ) и наблюдаемый профиль с шумом: ~18 % заявленных навыков приукрашены, ~15 % реальных не заявлены, категория — из теста с ошибкой измерения, часть навыков подтверждена тестом. Релевантность (0–3) считается по истине функцией, отличной от скоринга. Релевантным считается оценка ⩾ 2.

Ранжирование	Precision@10	nDCG@10	MRR
Наш подбор (веса после валидации)	0,941 ± 0,017	0,835 ± 0,023	1,000
Наш подбор, исходные веса	0,888 ± 0,022	0,758 ± 0,027	0,986
Ключевые слова резюме + заявленный стаж	0,425 ± 0,030	0,356 ± 0,028	0,632
Случайный порядок в специализации	0,223 ± 0,025	0,173 ± 0,018	0,414

Абляции: без результата теста nDCG падает до 0,819, без графа близости навыков — до 0,829, только по навыкам — до 0,739. Исходные веса (обязательные навыки 0,30, тест 0,25) проигрывали; на трёх случайных сидах устойчиво лучшим оказался вариант с большим весом стека и грейда — он и принят.

6.4. Проверка на своих данных

Главный критерий подбора — доля релевантных кандидатов в топе на наборе пар «вакансия – кандидат». Вводить такой набор через интерфейс долго, поэтому есть пакетный прогон: ваши вакансии, резюме и разметка проходят через тот же разбор текстов, тот же скоринг и ту же базовую линию, что в §6.3. Командная строка и метод API принимают один формат и возвращают один отчёт.

Входные данные — JSON с тремя списками или каталог с файлами `vacancies.csv`, `candidates.csv` и `labels.csv`. В CSV разделитель — запятая или точка с запятой, многострочные тексты берутся в кавычки, списки внутри ячейки разделяются `|`.

Список	Обязательно	Необязательно — если нет, распознаётся из текста
`vacancies`	`id`, `text` — текст вакансии, заголовок в первой строке	`specialization`, `grades`, `must_skills`, `nice_skills`, `salary_min`, `salary_max`, `work_formats`, `city`, `allow_adjacent_grades`
`candidates`	`id`, `resume_text` — текст резюме	`specialization`, `grade`, `skills`, `salary_expectation`, `work_formats`, `city`, `relocation`, `test_result` (0–100), `fsp_achievements` (модель §7.2)
`labels`	`vacancy_id`, `candidate_id`, `relevance` — 0–3	—

Значения справочников — слаги, как в `GET /api/v1/catalog`: `backend`, `middle`, `remote`. Лимиты прогона: до 200 вакансий, 5 000 кандидатов и 100 000 оценок, тексты до 20 000 символов. Пример — `docs/validation/pairs-example/`: 5 вакансий, 60 резюме и 65 оценок в JSON и CSV, рядом готовый отчёт.

Чего нет во входных данных, то не выдумывается, а допущения видны в отчёте:

- Специализация и грейд кандидата, если их нет, берутся из разбора резюме — того же, что при загрузке PDF, — и помечаются «из резюме, не подтверждена тестом». Грейд из резюме — заявленный или по стажу; если его нет нигде, принимается Middle с пометкой. Кандидат без распознанной специализации остаётся без категории и в наш подбор не попадает, базовая линия его видит.

- Без результата теста берётся нейтральное значение — середина диапазона грейда, как у типичного кандидата категории: отсутствие теста не помогает и не мешает. В объяснении написано «Результата теста нет», а не «тест пройден». `test_result` из входных данных — положение результата внутри грейда, 0–100.
- Соседний грейд по умолчанию допускается со штрафом признака «Грейд»: грейд из резюме не подтверждён тестом, и жёсткий фильтр по нему отсекал бы подходящих. Поле `allow_adjacent_grades` это меняет.
- Активности нет, поэтому признак «Актуальность» у всех нулевой и на порядок не влияет. Близость текстов в API считает PostgreSQL; здесь — доля основ из первых 40 значимых слов вакансии, найденных в резюме, нормированная на лучшего кандидата специализации, как в API.

Отчёт. Для каждой вакансии: как она распознана, топ-k нашего подбора с баллом, причинами, пробелами и источником категории, топ-k базовой линии «ключевые слова резюме + заявленный стаж» — той же, что в §6.3. При разметке считаются Precision@5, Precision@k, nDCG@k, MAP и MRR по каждой вакансии и в среднем по вакансиям, где есть хотя бы один релевантный кандидат. Релевантен кандидат с оценкой ⩾ 2, неразмеченная пара считается нерелевантной, доля размеченных в топе выводится рядом. Блок покрытия показывает, сколько полей взято из данных и сколько распознано, у скольких кандидатов нет теста и какие вакансии разобраны неуверенно.

Запуск из командной строки — отчёт в Markdown и JSON:

```
cargo run -p validate --release -- pairs --input data.json --out report.md
cargo run -p validate --release -- pairs --input каталог_с_csv --k 10 --now 2026-10-01
```

Или из Swagger (`/api/docs`, раздел «Валидация»): `POST /api/v1/eval/pairs` с тем же JSON, пример запроса уже подставлен. Метод публичный и ничего не сохраняет; тело до 8 МиБ, не больше 10 запросов за 10 минут с одного IP. Полный по лимитам набор — 200 вакансий и 5 000 резюме — командная строка обрабатывает за секунды.

На примере (разметка составлена до первого прогона):

Ранжирование	Precision@5	Precision@10	nDCG@10	MAP	MRR
Наш подбор	0,76	0,42	0,96	0,93	1,00
Ключевые слова резюме + заявленный стаж	0,68	0,42	0,85	0,82	0,80

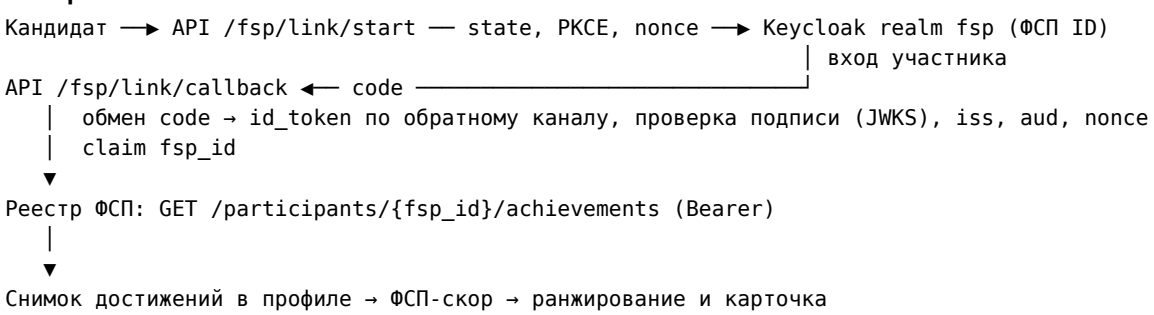
Precision@10 одинакова: релевантных на вакансию 3–6, и оба способа поднимают их в первую десятку; разница — в порядке внутри топа. Пример показывает и предел работы без тестов: выпускник курсов с полным списком ключевых слов стоит высоко в обоих ранжированиях, потому что без теста система видит только слова резюме. В рабочем режиме категорию и силу результата даёт тест.

6.5. Ограничения и экспертная проверка

Синтетическая «истина» задаётся командой, поэтому абсолютные цифры описывают поведение системы в заданных предпосылках, а не рынок. Сравнение с базовыми линиями и абляции от этого не зависят. Следующий шаг — экспертная разметка: 10–20 реальных вакансий партнёров ФСП, топ-20 выдачи каждой оценивают два эксперта по шкале 0–3; Precision@10 и nDCG@10 по такой разметке считает прогон из §6.4. Для тестов — пилот на участниках соревнований ФСП: сравнение грейда по тесту с разрядом и результатами соревнований, повторная сдача через неделю.

7. Интеграция с ФСП

7.1. Привязка ФСП ID



- ФСП ID работает на Keycloak; совместимость — стандартный OpenID Connect, клиент `fsp-talent` с обязательным PKCE S256. Claim `fsp_id` — атрибут пользователя Keycloak.

- Один ФСП ID можно привязать только к одному профилю. Отвязка удаляет снимок достижений.
- Если реестр недоступен, привязка сохраняется, а синхронизацию можно повторить из настроек. Достижения обновляются при привязке и по кнопке; ежедневная фоновая синхронизация — следующий шаг.
- Для интеграции с реальным реестром нужны его адрес и токен доступа. Сейчас API передаёт заглушке статический Bearer-токен (`FSP_REGISTRY_TOKEN`); получать токен у Keycloak по client credentials или token exchange — небольшая доработка клиента реестра.

7.2. Какие данные получаем

Для каждого достижения: тип (разряд или соревнование), дисциплина (алгоритмическое, продуктовое программирование, программирование систем ИБ, систем ИИ, робототехники, БАС), разряд (юношеский, III–I, КМС, МС), уровень соревнования (региональное, межрегиональное, всероссийское, международное), место (1–3, финалист, участник), команда, роль, дата, ссылка на протокол.

7.3. ФСП-скор

Вклад достижения = вес уровня × вес места (или вес разряда) × релевантность дисциплины специализации × затухание с периодом полураспада 2 года. Итог — «шумное ИЛИ»: $1 - \prod(1 - v_i)$. Скор растёт с каждым достижением, но насыщается и не может перевесить тест и стек (вес в подборе — 0,10). Учитывается только высший разряд в дисциплине.

Кандидат без истории ФСП получает 0 без штрафа: в карточке нейтральное «Нет данных ФСП», отсутствие ФСП никогда не попадает в «пробелы». Кандидат может скрыть ФСП-данные — тогда они не участвуют и в ранжировании.

8. Безопасность и 152-ФЗ

8.1. Аутентификация

- Пароли — Argon2id (19 МиБ, t = 2, p = 1); при неизвестном email выполняется та же проверка, чтобы время ответа не раскрывало наличие аккаунта.
- Короткий access-токен (JWT EdDSA, 15 минут) и ротируемый refresh-токен (30 дней) в httpOnly-cookie; повторное использование старого refresh-токена отзывает всю цепочку. Выход заносит access-токен в deny-list.
- Защита от CSRF: изменяющие запросы с cookie принимаются только с собственного Origin.
- Ограничение частоты: вход, регистрация, повторные письма, приглашения, жалобы.

8.2. Персональные данные

- Email, ФИО и контакты кандидата хранятся зашифрованными (XChaCha20-Poly1305); поиск по email — через keyed-хеш (blind index).
- Версионированные согласия на обработку и публикацию с фиксацией времени, IP и User-Agent. Каждый документ знает, кому он адресован: обработка и соглашение — обеим ролям, публикация и видеозапись — только кандидату.
- **Публикация только с согласием.** Показать профиль в поиске можно лишь при действующих согласиях на обработку и на публикацию; иначе сервер отвечает 409 `consent_required` и называет документ, который нужно принять, — интерфейс тут же предлагает его текст. Отзыв любого из них сразу скрывает профиль. Дать согласие снова можно в настройках: принимается только действующая редакция документа, повторная выдача ничего не дублирует.
- **Двойная проверка в поиске.** Пул кандидатов для подборок, поиска по банку, статистики категорий, анонимного профиля и приглашений сам проверяет действующее согласие на публикацию — флага «видим» недостаточно. Пул кешируется в памяти, но отзыв согласия, скрытие профиля и любая другая правка, меняющая выдачу, сбрасывают кеш до ответа. Сохранённая подборка и страница поиска перепроверяются по базе при каждом показе, поэтому скрытый профиль исчезает сразу, без пересборки и на любой копии сервера.
- **Ничего личного в кешах.** Ответы с персональными данными и всё, что требует входа или API-ключа, помечены `Cache-Control: private, no-store`: их не сохранит ни браузер на диск, ни прокси, ни CDN. Публично кешируются только справочники и опубликованные вакансии (§12.4).
- **Право на доступ (ст. 14).** Кнопка «Скачать мои данные» отдаёт JSON-файл со всем, что хранится об аккаунте: профиль, согласия с редакциями и датами, тесты с итогами (без самих заданий и ответов), история категории, приглашения, отклики, журнал доступа к контактам, сводки видеозаписи без видео. Работодатель получает свой аккаунт и данные компании.

- **Журнал доступа к контактам**: кто, когда, на каком основании. Кандидат видит его в настройках как «Кто открывал ваши контакты» — компания, время, основание (принятое приглашение или собственный отклик), без сотрудника и его IP.
- **Передача в ATS компании** — только после принятия приглашения или отклика и тем же путём, что и в кабинете; каждая передача видна кандидату в том же журнале. Дальше компания обрабатывает данные в своей ATS как самостоятельный оператор (§10.2).
- **Политика обработки персональных данных** (ст. 18.1) — публичная страница `/privacy`: состав данных, цели, основания, кому видны, сроки, cookies, права и как ими пользоваться в кабинете, действующие тексты согласий.
- Удаление аккаунта стирает персональные данные, связи обезличиваются.
- Хранение — на сервере в РФ.

Соискатели 14–17 лет — наше дополнение: среди участников ФСП много школьников, поэтому правило «18+ или согласие законного представителя».

- **Возраст при регистрации — одним вопросом.** «Мне есть 18 лет» — и больше ничего: дату рождения у взрослых не спрашиваем, ответ сохраняется вместе с выдачей согласий. «Мне меньше 14» — доброе сообщение, что регистрация открывается с 14 лет; аккаунт не создаётся. «Мне 14–17 лет» — дата рождения (сервер проверяет, что это действительно 14–17), ФИО и email законного представителя. Компания при регистрации подтверждает, что её представителю есть 18.
- **Письмо представителю** уходит, только когда подросток подтвердил свою почту. В письме две ссылки: ответить (одноразовая, 7 дней) и отозвать согласие (работает, пока согласие действует). Обе ведут на публичную страницу `/guardian/confirm`: кто мы, какие данные, что компании видят анонимную карточку, контакты — только после того, как подросток сам примет приглашение, видео — только с отдельного согласия, как отозвать. Повторная отправка и смена адреса из кабинета выпускают новые ссылки и гасят старые; лимит — 3 письма в час на подростка и 5 в сутки на один адрес.
- **Согласие представителя** — отдельный версионированный документ `guardian_consent`: выдача записывается на подростка с ФИО и email представителя (зашифрованы, у email — blind index), временем, IP и браузером. Сам подросток выдать или отозвать его не может.
- **Публикация.** Пока согласия нет, кабинет и тест доступны, но профиль не показывается: `PUT /candidate/privacy` отвечает 409 `guardian_consent_required`, а пул поиска сам проверяет согласие представителя у всех, кому нет 18, — та же двойная проверка, что и для согласия на публикацию. Отклик на вакансию и принятие приглашения, которые открывают компании контакты, до согласия тоже недоступны. Отказ или отзыв сразу скрывает профиль, а баннер в кабинете объясняет почему и предлагает отправить письмо ещё раз или указать другой адрес.
- **18 лет — без расписаний.** «Несовершеннолетний» вычисляется из даты рождения в момент запроса, поэтому в день 18-летия ограничения снимаются сами.
- **Работодатель** видит нейтральную отметку «14–17 лет» (дату рождения — никогда), в окне приглашения — короткую памятку об ограничениях Трудового кодекса для работников до 18 лет (ст. 63, 92, 94, 265), в поиске по базе и в подборке — фильтр «Только 18+». Это фильтр, а не понижение в выдаче.
- **Срок хранения.** Через 30 дней после 18-летия ФИО, email, IP и браузер представителя и сама дата рождения стираются; остаётся факт: редакция согласия, время выдачи и отзыва. Очистка запускается при старте сервера и каждые 6 часов. При удалении аккаунта данные представителя стираются сразу. Выгрузка данных содержит возрастную группу, дату рождения подростка и записи о согласии представителя.

Видеозапись теста (§4.7):

- **Согласие** — отдельный версионированный документ «Согласие на видеозапись во время теста», при регистрации не запрашивается. Его выдача фиксируется так же, как остальные: время, IP, User-Agent. Без согласия запись не начинается: сервер отклоняет старт теста с камерой, если не подтверждена текущая версия документа.
- **Что именно:** только изображение с камеры, без звука и содержимого экрана; запись не используется для распознавания лица и установления личности.
- **Шифрование.** Каждый 10-секундный фрагмент шифруется XChaCha20-Poly1305 тем же модулем, что и персональные данные, и привязывается к своей сессии и номеру: файл, перенесённый в чужую запись или на другое место, не расшифруется. Файлы лежат в отдельном каталоге (в Docker — отдельный том), в базе — только метаданные: выбор камеры, ссылка на согласие, номера и размеры фрагментов, события. Запись отдаётся потоком и расшифровывается по одному фрагменту.

- **Срок хранения** — 90 дней. Фоновая задача удаляет просроченные записи при старте сервера и каждые 6 часов; сводка и отметка остаются.
- **Отзыв согласия** в настройках сразу удаляет все записи кандидата; **удаление аккаунта** удаляет их вместе с остальными данными.
- **Лимиты:** фрагмент до 1 МиБ, запись до 64 МиБ, не больше 50 событий за запрос и 500 на тест.

8.3. Разграничение доступа

- Роли кандидата и работодателя проверяются на уровне маршрутов; кандидат видит только свои приглашения, отклики и тесты, чужие ресурсы возвращают 404.
- **Контакты закрыты типами.** Карточка кандидата для работодателя не содержит полей контактов. Контакты можно получить только из «основания доступа», которое выдаёт единственная функция: она проверяет принятое приглашение или отклик и пишет журнал.
- **Видео закрыто так же.** В карточке для работодателя есть только отметка о видеозаписи — статус и число замечаний, без идентификатора сессии и ссылок. Запись отдают два метода: кандидату — его собственная, администратору — любая, с записью в журнал. Для работодателя оба отвечают 403: анонимность сохраняется до принятия приглашения, а видео не раскрывается и после.

8.4. ГОСТ и сертифицированная криптография

Алгоритм хранится вместе с данными (строка хеша PHC, версия шифротекста), хеширование и шифрование — за интерфейсами. Переход на ГОСТ (PBKDF2-HMAC-Стрибог по Р 50.1.111-2016, «Кузнечик») и сертифицированное СКЗИ — это новая реализация интерфейса без изменения схемы данных; в MVP ГОСТ не используется, так как соответствие обеспечивает сертификация СКЗИ, а не сам алгоритм.

9. API

Все методы описаны в OpenAPI 3.1: спецификация `/api/v1/openapi.json`, Swagger UI — `/api/docs`. 116 операций на 98 путях.

- Версионирование: префикс `/api/v1`; несовместимые изменения — новая версия рядом.
- Формат: JSON, даты RFC 3339, деньги — целые рубли.
- Ошибки приложения — RFC 9457 `application/problem+json` со стабильным машинным полем `code` (например, `grade_change_cooldown`, `invitation_quota_exceeded`, `contacts_not_granted`, `consent_required` — с полем `document_id` документа, который нужно принять); 404 и для чужих ресурсов; 409 — конфликт состояния; 422 — ошибки полей; 429 — лимит.
- Аутентификация: httpOnly-cookie или заголовок `Authorization: Bearer`. API для ATS работодателя — только API-ключ компании `Bearer fsp_live_…` со скоупами.
- Служебные пути вне спецификации: `/healthz` (живость), `/readyz` (база и Valkey), `/metrics` (Prometheus, наружу не отдаётся). Соглашения и полный список с ролями и кодами ошибок — `api.md` в репозитории.

Группа	Что делает	Операций
`auth`	Регистрация с согласиями, подтверждение email, вход, обновление и завершение сессии	6
`account`	Текущий пользователь, согласия: выдача, отзыв, повторная выдача; выгрузка всех данных, удаление аккаунта, жалобы	7
`catalog`	Справочники, автодополнение навыков, документы согласий	3
`candidate`	Профиль, PDF-резюме и PDF-профиль, приватность, журнал доступа к контактам, опрос, статистика спроса; для 14–17 лет — статус согласия представителя, повторное письмо, смена адреса	13
`assessment`	Старт теста, текущее задание, ответ, история, категория, план теста под вакансию	8
`proctoring`	Шаг перед тестом, фрагменты видеозаписи, журнал событий, просмотр своей записи; для администратора — журнал и запись любой сессии	6
`fsp`	Привязка ФСП ID (OIDC), dev-привязка, достижения, синхронизация, отвязка	6
`employer`	Профиль компании	2
`matching`	Потребности, распознавание текста вакансии, категории, подборка, поиск по банку, анонимный профиль	13
`invitations`	Приглашения с обеих сторон, квота и статистика, контакты	10
`vacancies`	Вакансии, публичный список, отклики	11
`tasks`	Регулярные задания	8
`guardian`	Публичная страница законного представителя по ссылке из письма: что он подтверждает; ответ — подтвердить, отказаться, отозвать	2
Валидация	Проверка подбора на своём наборе вакансий, резюме и разметки (§6.4)	1
`trust`	Уровень доверия и квота компании с причинами, проверка вилки по рынку категории, поиск похожих собственных вакансий; для администратора — очередь модерации	4
`integrations`	Кабинет «Интеграции»: вебхук, смена секрета, тестовое событие, журнал доставок и повтор, API-ключи	10
`Интеграции ATS`	API для ATS по ключу: опрос приглашений, профиль и PDF после согласия, этапы найма, импорт вакансий	6

10. Антифейк и концепции вне MVP

10.1. Защита от фиктивных вакансий и недобросовестных работодателей

ТЗ относит антифейк к концепциям вне MVP. У нас работает его основная часть. Главный принцип — **ни один сигнал не блокирует публикацию или приглашение**: эвристика ошибается, и честная компания, которая платит выше рынка, не должна от этого страдать. Сигнал делает три вещи: мягко предупреждает работодателя в форме, учитывается в уровне доверия к компании (и через него в квоте приглашений) и попадает в очередь модерации.

Базовые ограничения работали и раньше: вилка «от–до» обязательна, не больше одного действующего приглашения на пару «компания–кандидат», приглашение живёт 14 дней, кандидат может пожаловаться на вакансию, приглашение или компанию.

Сигнал	Правило	Работодатель видит	Кандидат видит
Корпоративный домен	Подтверждённая почта для входа и сайт компании на одном домене; поддомены подходят в обе стороны (`hr@corp.ru` и `jobs.corp.ru`). Публичные почтовые сервисы не засчитываются никогда: gmail.com, yandex.ru, ya.ru, mail.ru, bk.ru, inbox.ru, list.ru, rambler.ru, outlook.com, icloud.com, proton.me и другие, включая их поддомены. Общие зоны (`com.ru`, `msk.ru`, `github.io`, `tilda.ws` и т. п.) и голые TLD не считаются доменом компании. Домены сравниваются в ASCII: `пример.рф` и `xn--e1afmkfd.xn--p1ai` — один сайт	Статус и подсказку, как получить отметку: какой домен у почты, какой у сайта, что поправить	Нейтральную отметку «Домен подтверждён» у вакансии и приглашения. Если отметки нет, не показываем ничего
Вилка выше рынка	Верх вилки больше 1,5 × 90-го перцентиля ожиданий кандидатов категории. Предложения обычно на 10–30 % выше ожиданий, поэтому щедрая, но реальная вилка под порог не попадает. Если в категории меньше 5 указанных ожиданий, проверка не выполняется	Предупреждение в форме вакансии и приглашения, рядом медиана и 90-й перцентиль рынка категории	Ничего
Слишком широкая вилка	Верх ⩾ 3 × низ: это разброс от джуна до синьора, а не одна позиция (классика фейка — «от 50 до 500 тысяч»)	То же	Ничего
Дубль своей вакансии	Пары соседних слов текста (название + описание) сжимаются в MinHash-подпись из 128 хешей; доля совпавших хешей оценивает долю общих пар слов. Порог — 0,6: правки и перестановка абзацев оставляют сходство выше 0,75, две разные вакансии одной компании с общим блоком «о компании» дают около 0,2	Перед сохранением: «Похожая вакансия уже есть» со ссылкой — предложение обновить её; сохранить всё равно можно	—
Копия чужой вакансии	То же сходство с опубликованной за 90 дней вакансией другой компании	В уровне доверия: «вакансия почти дословно совпадает с вакансией другой компании»	—
Поведение	Жалобы кандидатов (уникальные авторы, ещё не закрытые модератором, за 90 дней); доля отказов с причиной «зарплата» или «компания»; доля отозванных приглашений; доля приглашений, оставшихся без ответа	В уровне доверия с цифрами	—

Почему кандидат не видит подсказку о вилке. Высокая вилка сама по себе не улика: компания может честно платить больше рынка, а наша статистика — ожидания кандидатов, а не реальные предложения. Нейтральная фраза «Вилка выше обычной для этой категории» на практике читается либо как тревога и бьёт по честной компании, либо как приманка и помогает фейку. Поэтому сигнал показываем там, где с ним можно что-то сделать: работодатель исправит опечатку прямо в форме, модератор увидит вакансию в очереди, а квота снизится, если такие вилки повторяются.

Уровень доверия и квота приглашений. Правило одинаковое для всех и видно работодателю целиком:

- новая компания начинает с 10 приглашений в неделю;
- +10 за подтверждённый домен, +10 после 30 дней на платформе, +10, когда кандидаты примут 3 приглашения (+5 — после первого принятого). Бонусы складываются до +20: любых двух шагов хватает до максимума 30, так что компания с публичной почтой тоже может его достичь;

- −10 за каждую жалобу кандидата за 90 дней (не больше −30); −10, если у 40 % и больше отказов (при 5+ отказах) причина «зарплата не соответствует» или «компания»; −5 за каждую вакансию или приглашение с вилкой выше рынка или слишком широкой (не больше −10); −20, если есть вакансия, скопированная у другой компании;
- −5, если отозвано 30 % и больше из 5+ приглашений; −5, если кандидаты ответили на 10 % и меньше из 10+ закрытых приглашений; −5, если есть копии собственных вакансий;
- квота не опускается ниже 5 — одна ошибка или одна несправедливая жалоба не останавливает работу. Отметки на вакансиях перестают учитываться через 90 дней.

Уровень — для подписи: «Новая компания» (меньше месяца, замечаний нет), «Обычный», «Высокий» (квота 30) и «Квота временно снижена» (есть замечания и квота опустилась до 10 и ниже). В кабинете работодателя раздел «Доверие и квота» показывает текущую квоту как сумму «база + бонусы − замечания», каждую причину с её вкладом — так же, как объяснение балла в подборке, — и конкретные шаги, которые её поднимут. Формулировки без обвинений: «Есть замечания — они перестают учитываться через 90 дней».

Модерация. `GET /api/v1/admin/trust` (роль администратора) отдаёт компании хотя бы с одним отрицательным сигналом — от самой низкой квоты, вакансии и приглашения с отметками и причинами словами («Верх вилки 900 000 ₽ — в 3,1 раза выше 90-го перцентиля ожиданий Бэкенд-разработка · Middle»). Отметки хранятся вместе с вакансией и приглашением (JSON с видом сигнала и цифрами на момент публикации), подпись текста — 512 байт в строке вакансии, поэтому новая вакансия сравнивается с тысячами недавних за доли миллисекунды без отдельного индекса.

Логика — чистый модуль `matching::trust` без ввода-вывода и часов; пороги и правило квоты покрыты модульными тестами, запись отметок при публикации и приглашении и изменение квоты — интеграционными тестами на живой базе.

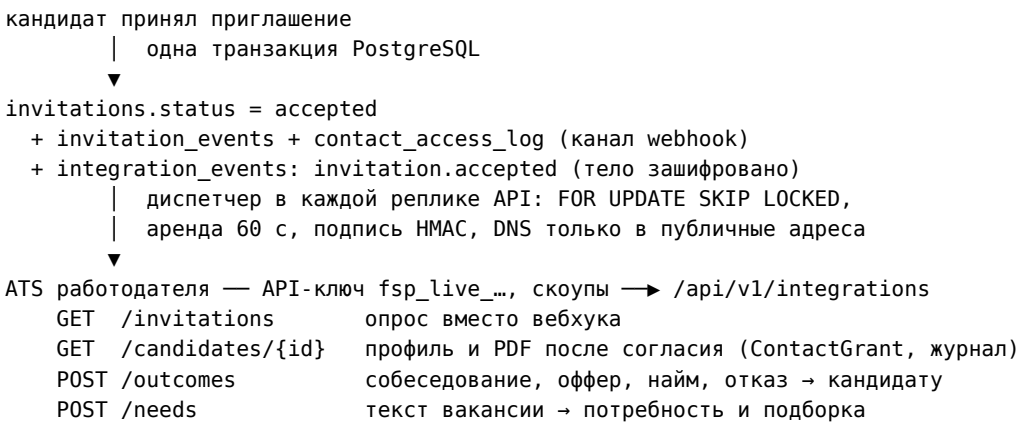
Следующие шаги.

- Проверка ИНН и ОГРН по ЕГРЮЛ (открытый сервис ФНС): название, статус «действующее», дата регистрации; несовпадение названия или ликвидированное юрлицо — сильный отрицательный сигнал.
- Подтверждение домена DNS-записью TXT — для компаний, которые не могут войти с корпоративной почты.
- Модерация с решениями: снять отметку, подтвердить или отклонить жалобу, временно закрыть приглашения; каждое решение — в журнал и в уровень доверия.
- Отчётность: доля отмеченных вакансий, время реакции модератора, жалобы на 100 приглашений — по неделям.
- Эмбеддинги текста (pgvector) для пересказанных копий: MinHash ловит правки и перестановки, но не перефразирование.

10.2. Интеграция с ATS работодателя

ТЗ относит интеграцию с внутренними ATS к концепциям. Мы сделали её рабочим универсальным слоем, через который подключается любая система подбора, умеющая принять HTTPS-запрос или опрашивать REST API. Нативные коннекторы к конкретным ATS — тонкие адаптеры поверх этого слоя, их план — в конце раздела. В демо-стенде компания `employer@demo.fsp` уже подключена к демо-ATS (`http://localhost:8082`): события о приглашениях приходят туда с подписью, а кнопки «Собеседование», «Оффер», «Нанят», «Отказ» возвращают этап кандидату.

Как устроено



Часть	Что делает
Outbox событий	Событие записывается в той же транзакции, что и смена статуса. Если запись не удалась, откатываются и статус, и запись в журнале доступа — состояние платформы и поток событий не расходятся
Диспетчер	Фоновая задача в каждой реплике API. Берёт до 16 готовых событий с `FOR UPDATE SKIP LOCKED` и сразу сдвигает время следующей попытки на 60 секунд: реплики не отправят одно событие дважды одновременно, а упавшая реплика его не потеряет. Повторы через 1 мин, 5 мин, 30 мин, 2 ч и 12 ч, пока очередной укладывается в сутки от создания; дальше событие помечается недоставленным (у адреса, который не отвечает, — после шестой попытки, примерно через 14,6 ч)
Вебхук компании	Один HTTPS-адрес на компанию. Секрет подписи показывается один раз; его можно сменить (сутки события подписаны обоими секретами) или отключить вебхук целиком
Журнал доставок	Каждая попытка: код ответа, задержка, ошибка. Кнопки «Отправить ещё раз» и «Тестовое событие» — в кабинете «Интеграции»
API-ключи	`Authorization: Bearer fsp_live_…`, скоупы `invitations:read`, `candidates:read`, `outcomes:write`, `needs:write`. Ключ работает только в дереве `/api/v1/integrations`
Этапы найма	ATS отмечает собеседование, оффер, найм или отказ. Кандидат видит «Компания отметила: собеседование» в приглашении или отклике, работодатель — этапы в воронке на главной
Импорт вакансий	Текст вакансии из ATS разбирается тем же разборщиком, что и форма потребности: специализация, грейды, стек, вилка, формат. В ответе — id потребности и ссылка на подборку

Каталог событий. Все события приходят одним конвертом `{id, type, created_at, company_id, data}`; `id` совпадает с заголовком `FSP-Event-Id`. Доставка гарантирована «хотя бы один раз», поэтому ATS отбрасывает повторы по этому id.

Событие	Когда	Что о кандидате
`invitation.sent`	компания отправила приглашение	анонимный код и категория
`invitation.viewed`	кандидат открыл приглашение	анонимный код и категория
`invitation.accepted`	кандидат принял	id, ФИО, телефон, Telegram, email, ссылки на профиль и PDF
`invitation.declined`	кандидат отклонил	анонимный код и категория; причина и комментарий
`invitation.expired`	прошло 14 дней без ответа	анонимный код и категория
`invitation.withdrawn`	компания отозвала	анонимный код и категория
`application.created`	кандидат откликнулся на вакансию	id, ФИО и контакты: кандидат откликнулся сам
`application.status_changed`	отклик сменил статус	id, ФИО и контакты; после отзыва отклика — только код
`contacts.opened`	сотрудник открыл контакты в кабинете	id, основание и время, без самих контактов
`integration.test`	кнопка «Тестовое событие»	—

До принятия:

```
{
  "id": "0199b1c2-7f3e-7c3a-9d1e-2f6a8b0c4d11",
  "type": "invitation.sent",
  "created_at": "2026-10-06T09:12:44Z",
  "company_id": "0199a0f1-…",
  "data": {
    "invitation": {
      "id": "0199b1c2-…", "status": "sent",
      "title": "Backend-разработчик в платёжную команду",
      "salary": { "min": 280000, "max": 340000 }, "work_format": "remote",
      "expires_at": "2026-10-20T09:12:44Z", "updated_at": "2026-10-06T09:12:44Z"
    },
    "candidate": {
      "code": "K-7F3A2C",
      "category": { "specialization": "backend", "grade": "middle" }
    }
  }
}
```

После принятия у того же приглашения:

```
"candidate": {
  "code": "K-7F3A2C",
  "category": { "specialization": "backend", "grade": "middle" },
  "id": "0199a3d0-…",
  "disclosed": {
    "full_name": "Анна Смирнова", "phone": "+7 900 123-45-67",
    "telegram": "@anna_dev", "email": "anna@example.ru", "basis": "invitation",
    "profile_url": "https://…/api/v1/integrations/candidates/0199a3d0-…",
    "profile_pdf_url": "https://…/api/v1/integrations/candidates/0199a3d0-…/profile.pdf"
  }
}
```

Этап найма из ATS обратно:

```
POST /api/v1/integrations/outcomes
Authorization: Bearer fsp_live_…

{ "invitation_id": "0199b1c2-…", "stage": "interview",
  "reason": "Зовём на техническое интервью", "at": "2026-10-08T15:00:00+03:00" }
```

Повтор того же этапа обновляет причину и время, а не создаёт дубль — ATS может безопасно повторять запрос. Этап по непринятому приглашению отклоняется (409): до согласия кандидата компании нечего отмечать.

Безопасность

- **Подпись.** Заголовок `FSP-Signature: t=<unix>,v1=<hex>`, где `v1 = HMAC-SHA256(секрет, t + "." + тело)`. Метка времени входит в подпись, а получатель отклоняет запросы старше 5 минут, поэтому перехваченное событие нельзя отправить повторно позже. Секрет хранится зашифрованным и привязан к компании.
- **Защита от SSRF.** Адрес компании — это ввод пользователя, и платформа не должна стать прокси во внутреннюю сеть. Принимаются только `https://` без логина в адресе, редиректы не выполняются. Адрес проверяется внутри DNS-резолвера HTTP-клиента при каждом соединении: непубличные адреса имени (loopback, частные сети, link-local с 169.254.169.254, CGNAT, служебные диапазоны, включая IPv6 и IPv4 внутри IPv6) отбрасываются, соединение идёт только на публичные, а имя без публичных адресов отклоняется. Поэтому имя, которое после проверки начали резолвить в 127.0.0.1 (DNS rebinding), во внутреннюю сеть не попадёт. Исключение — хосты из `WEBHOOK_DEV_HOSTS`: в демо так подключён контейнер демо-ATS.
- **Ключи.** `fsp_live_` и 256 случайных бит. В базе только SHA-256 и видимый префикс: при такой энтропии медленный хеш не нужен, а по утёкшей базе ключ не восстановить. Ключ показывается один раз, отзывается мгновенно, у каждого видно время последнего запроса. Лимит — 120 запросов в минуту на ключ.
- **Скоупы и границы.** Ключ без `candidates:read` не получит профиль, без `outcomes:write` не отметит этап. Дерево `/api/v1/integrations` принимает только ключ — cookie и JWT туда не доходят, а с ключом не открыть ни один метод кабинета.

152-ФЗ

- **Что и когда уходит в ATS.** До принятия приглашения — только анонимный код, категория и данные самого приглашения: должность, вилка, формат, статус, причина отказа. ФИО, контакты и PDF-профиль попадают в событие только после того, как кандидат принял приглашение или сам откликнулся. Это то же основание, по которому компания видит контакты в кабинете, и в коде путь тот же: данные раскрываются только через `ContactGrant`. Видео теста, ответы на задания и данные других компаний не уходят никогда.
- **Каждая передача в журнале.** Кандидат видит в «Кто открывал ваши контакты» и открытие в кабинете, и передачу в ATS компании — с пометкой «передано в систему подбора компании».
- **Кто отвечает дальше.** Получив данные после согласия кандидата, компания обрабатывает их в своей ATS как самостоятельный оператор — для своего подбора, по своим правилам хранения. Об этом сказано в политике обработки `/privacy`. Отключение вебхука или отзыв ключа прекращает передачу; удалить уже переданное кандидат может, обратившись к компании.
- **Минимизация у нас.** Тело события в очереди зашифровано тем же XChaCha20-Poly1305, что и поля ПДн, с привязкой к id события. Завершённые события удаляются через 30 дней, а при удалении аккаунта кандидата его события удаляются сразу. Компания без вебхука событий не получает вовсе, значит и раскрытия нет.

Этапы найма как разметка. Главный пробел валидации подбора — нет экспертной разметки, кто из подборки на самом деле подошёл (§6.4). Этапы из ATS — это настоящие исходы найма по конкретным приглашениям: дошёл до собеседования, получил оффер, вышел на работу. Они хранятся вместе с приглашением, а оно связано с потребностью и подборкой, где у кандидата был свой балл. Накопив исходы, можно калибровать веса ранжирования на реальных данных, а не на синтетике: подбирать веса признаков так, чтобы нанятые оказывались выше в подборке.

Нативные коннекторы. Универсальный слой уже покрывает ATS, которые принимают вебхуки или умеют опрашивать REST. Нативный коннектор — отдельный воркер, который читает тот же outbox как внутренний вебхук и переводит события в API конкретной системы. Его настройки в кабинете: токен ATS (хранится зашифрованным), вакансия по умолчанию и таблица соответствия статусов компании нашим этапам. Сведения о Huntflow, «Потоке» и Talantix взяты из их опубликованной документации API; про E-Staff и Skillaz открытой спецификации мы не нашли, для них описан подход.

Система	Авторизация	Мы → ATS (после согласия кандидата)	ATS → мы (этапы найма)
Huntflow	Bearer-токен из настроек Huntflow: `access_token` живёт 7 дней, `refresh_token` — 14, обновление `POST /v2/token/refresh`	`POST /accounts/{account_id}/applicants` (ФИО, телефон, email, соцсети), `POST /accounts/{account_id}/upload` (PDF), `POST /accounts/{account_id}/applicants/{applicant_id}/vacancy` (на вакансию, со статусом и файлами)	вебхук Huntflow `POST /accounts/{account_id}/hooks` с событиями `APPLICANT`; статусы — `GET /accounts/{account_id}/vacancies/statuses`
Поток	Токен из раздела «Генерация токенов»: `Authorization: Bearer …` или параметр `token`; база `https://app.potok.io/api/v3`; лимит 100 запросов за 10 с	`POST /applicants.json` (кандидат), `POST /applicants/{applicant_id}/resume_file.json` (PDF), `POST /ajs_joins.json` (на вакансию)	исходящих вебхуков в публичной спецификации v3 нет — опрос истории `GET /jobs/{job_id}/events.json` и карточки `GET /applicants/{applicant_id}.json`
Talantix (hh.ru)	Токен из личного кабинета Talantix; GraphQL API	мутация `createPerson`, перевод на этап воронки `transferPersonsToWorkflowStatus`	исходящих уведомлений в документации мы не нашли — периодический опрос GraphQL API
E-Staff	По договорённости с вендором; часто стоит внутри периметра компании	ATS сама забирает данные: `GET /integrations/invitations` и `GET /integrations/candidates/{id}` — входящие соединения из интернета не нужны	`POST /integrations/outcomes` с ключом
Skillaz	По договорённости с вендором	вебхук на адрес интеграции Skillaz или опрос нашего API	`POST /integrations/outcomes` с ключом

Соответствие полей и статусов — одно для всех коннекторов:

У нас	Huntflow	Поток	Talantix
ФИО, телефон, email, Telegram	`last_name`, `first_name`, `middle_name`, `phone`, `email`, `social`	карточка кандидата `POST /applicants.json`	персона (`createPerson`)
PDF-профиль	файл через `upload`, затем `files` при добавлении на вакансию	файл резюме `resume_file.json`	ссылка на PDF в комментарии к персоне (`createPersonComment`)
Потребность / вакансия	вакансия `POST /accounts/{account_id}/vacancies`	вакансия `POST /jobs.json`, потребность `POST /demands.json`	заявка на подбор `createHiringRequest`
`interview`, `offer`	пользовательские статусы компании — по таблице соответствия в кабинете	этап воронки (`stage_type`) — по таблице	этап воронки (`workflowStatus`) — по таблице
`hired`	статус `hired` (с квотой и датой выхода)	статус трудоустройства финалиста	найм персоны (`hirePersons`)
`rejected`	статус `rejected` с причиной отказа (`rejection_reasons`)	отклонение `POST /jobs/{job_id}/{applicant_id}/decline.json`	отказ персоне (`discardPersons`)

Источники: Huntflow API v2 (спецификация `https://api.huntflow.ru/v2/openapi.json`), публичное API «Потока» v3 и описание на сайте, документация API Talantix и анонс API Talantix 2.0 на hh.ru.

10.3. Масштабирование

API без состояния за балансировщиком; общие лимиты и поколение кешей в Valkey; реплика PostgreSQL на чтение; эмбеддинги текстов (pgvector) — отдельным воркером; аналитика событий — выгрузка изменений в ClickHouse.

11. Библиотеки и компоненты

Весь код и зависимости — с открытыми лицензиями (MIT, Apache-2.0, BSD, ISC, MPL, OFL для шрифтов). Версии — из `Cargo.lock` и `web/package-lock.json`. Исключение — GSAP: распространяется бесплатно, в том числе для коммерческого использования, по собственной Standard License, которая не входит в число лицензий, одобренных OSI. GSAP отвечает только за анимации публичного сайта: без него страницы работают, а при `prefers-reduced-motion` анимации отключаются.

Компонент	Версия	Назначение
Rust	1.85+ (проверено на 1.99)	Язык backend
axum / tokio / tower-http / axum-extra	0.8.9 / 1.53 / 0.6 / 0.12	HTTP-сервер, асинхронный рантайм, middleware, cookie
sqlx	0.8.6	PostgreSQL: пул, запросы с проверкой при компиляции, миграции
utoipa / utoipa-axum / utoipa-swagger-ui	6.0 / 0.3 / 10.0	OpenAPI и Swagger UI
argon2 / chacha20poly1305 / ed25519-dalek / jsonwebtoken / hmac / sha2	0.5.3 / 0.10.1 / 2.2 / 11.1 / 0.12 / 0.10	Криптография, токены, подписи вебхуков
redis (для Valkey)	1.2	Лимиты, deny-list, поколение кеша кандидатов
lettre	0.11	Почта
reqwest (rustls)	0.13	Запросы к ФСП ID и реестру, вебхуки в ATS
arc-swap	1.9	Снимки пула кандидатов в API и кольцевой буфер событий демо-ATS — без блокировок
krilla / pdf-extract	0.8.2 / 0.12.1	Генерация и разбор PDF
metrics-exporter-prometheus / tracing	0.18 / 0.1	Метрики и логи
regex / rustc-hash / rand / rand_chacha	1.13 / 2.1 / 0.9 / 0.9	Разбор текста, словари, детерминированная генерация
Next.js / React / TypeScript	16.3 / 19.2 / 5.9	Frontend
TanStack Query / openapi-typescript	5.104 / 7.13	Данные и типы клиента из OpenAPI
Tailwind CSS / react-hook-form / zod / lucide-react	4.3 / 7.89 / 4.6 / 1.52	Стили, формы, иконки
GSAP (ScrollTrigger, SplitText, DrawSVG, ScrambleText, Draggable, Inertia, Flip, MorphSVG, MotionPath) / `@gsap/react` / Lenis	3.15 / 2.1 / 1.3	Анимации и плавный скролл публичного сайта
PostgreSQL (+pgvector) / Valkey	17 / 8	Хранилище и кэш
Keycloak	26.7	ФСП ID (OIDC)
Caddy / Mailpit	2.11 / 1.31	Входной прокси с TLS, почта для демо
Prometheus / Loki / Alloy / Grafana	3.5 / 3.7 / 1.20 / 12.4	Наблюдаемость
JetBrains Mono	2.304	Фирменный шрифт ФСП: подписи и цифры интерфейса, PDF-профиль
Onest	Google Fonts через `next/font`	Шрифт текста и заголовков сайта

12. Сборка, развёртывание и запуск

12.1. Docker (рекомендуется)

Требования: Docker с Compose v2, 4 ГБ свободной памяти.

```
git clone https://github.com/K1NGSM4N3/fsp-hack && cd fsp-hack
cp .env.example .env
docker compose -f deploy/compose.yml --profile full up -d --build   # или make up
```

Что	Адрес
Приложение	http://localhost:8080
Swagger UI	http://localhost:8080/api/docs
Письма (подтверждение email, письма представителю)	http://localhost:8025
ФСП ID (Keycloak), админка — `/admin`	http://localhost:8081
Демо-ATS работодателя	http://localhost:8082
Grafana (дашборд API)	http://localhost:3001

Профили compose: ядро (Caddy, web, API, PostgreSQL, Valkey, Mailpit) поднимается всегда, `fsp` добавляет Keycloak и заглушку реестра ФСП, `ats` — демо-ATS, `obs` — Prometheus, Loki, Alloy и Grafana, `full` — всё сразу. Наружу открыты только порты из таблицы; API, web, база и Valkey доступны лишь внутри сети compose, `/metrics` через Caddy закрыт.

При первом старте API применяет миграции и заполняет демо-данные (`SEED_DEMO=true`, только на пустой базе): 6 вымышленных компаний, 12 вакансий, 240 кандидатов с симулированными тестами, у 15 привязан ФСП ID (у 13 из них есть достижения, двое — без истории ФСП); у большинства тест пройден с видеозаписью (самих видеофайлов в демо нет). Шестеро кандидатов — 14–17 лет: у четырёх согласие представителя подтверждено, двое ждут ответа. У демо-работодателя шесть потребностей — по одной на специализацию, и он уже подключён к демо-ATS. Пять компаний из шести входят с почты на домене своего сайта, поэтому у их вакансий есть отметка «Домен подтверждён»; у демо-работодателя её нет, и раздел «Доверие и квота» показывает, как её получить.

Демо-доступы (пароль `demo12345`): работодатель `employer@demo.fsp`; работодатели с подтверждённым доменом `hr@pixel-manufactura.example`, `hr@oblakostroy.example`, `hr@medsignal.example`, `it-hr@logplus.example`, `career@shield-key.example`; кандидаты `candidate@demo.fsp`, `cand1@demo.fsp` … `cand239@demo.fsp`; кандидат 14–17 лет, ожидающий согласия представителя, — `minor@demo.fsp` (он занимает один из номеров `cand…`); участники ФСП в Keycloak — `fsp-000001` … `fsp-000010`. Роль администратора платформы регистрацией не выдаётся и демо-аккаунта не имеет.

12.2. Локальная разработка без Docker

Требования: Rust ⩾ 1.85, Node.js ⩾ 20.9, PostgreSQL ⩾ 14. Либо `nix develop` — окружение со всеми инструментами.

```
make db                  # база fsp_hack
SEED_DEMO=true make api  # API на :8000: миграции при старте, демо-данные; ФСП ID — dev-привязкой
make web                 # интерфейс на :3000, /api/* проксируется на :8000
make validate            # отчёты валидации в docs/validation/
make test                # тесты; тесты БД создают временные базы на сервере из TEST_DATABASE_URL
make docs                # этот документ в PDF и DOCX
```

Без Valkey лимиты и отозванные токены хранятся в памяти процесса, без SMTP ссылки из писем печатаются в лог API. Сборка не требует базы: запросы sqlx проверяются по метаданным из `.sqlx/`. После изменения SQL или миграций — `make db-prepare`.

12.3. Для VPS

- В `.env` — публичные адреса (`PUBLIC_URL`, `FSP_OIDC_ISSUER`, `FSP_OIDC_REDIRECT_URL`) и `COOKIE_SECURE=true`. Камера в браузере доступна только по HTTPS или на localhost; видеозапись отключается `PROCTORING=off`, файлы лежат в томе `proctoring-data`.
- Новые ключи `JWT_ED25519_SEED_B64`, `FIELD_ENC_KEY`, `BIDX_KEY` — по 32 случайных байта в base64 (`openssl rand -base64 32`), новые пароли PostgreSQL и Keycloak (`KC_BOOTSTRAP_ADMIN_*`), секрет клиента ФСП ID.
- Без демо: `SEED_DEMO=false`, пустые `WEBHOOK_DEV_HOSTS` и `ATS_DEMO_*` — тогда вебхуки уходят только по HTTPS на публичные адреса.
- `KC_HOSTNAME` (публичный адрес Keycloak) и `GRAFANA_ADMIN_USER` / `GRAFANA_ADMIN_PASSWORD` подставляются в `deploy/compose.yml` из окружения shell или из `deploy/.env`, а не из корневого `.env`.
- В `deploy/Caddyfile` — домен вместо `:8080` и без `auto_https off`, в `deploy/compose.yml` у `caddy` — порты 80 и 443: Caddy сам получит TLS-сертификат.

12.4. Производительность и кеширование

Пул кандидатов. Подборка, поиск по банку, рекомендованные категории и объяснение в профиле ранжируют всех кандидатов специализации. Раньше пул собирался из базы на каждый запрос; теперь каждая специализация хранится в памяти API компактным снимком, а запрос только фильтрует его и считает баллы. Снимок сбрасывается после любой записи, меняющей выдачу: профиль, навыки, категория, приватность, согласия, ФСП, удаление аккаунта. Между копиями сервера изменение доходит через счётчик в Valkey не дольше чем за 250 мс, страховочный срок жизни снимка — 30 с; отзыв согласия и скрытие профиля действуют с первого же запроса, потому что показанные карточки сверяются с базой. Ранжирование на тех же данных совпадает со старым путём (86 сравнений подборок, поиска и объяснений на 10 000 кандидатах). Подробно — architecture.md §12.2.

HTTP. Справочники — `public, max-age=300, stale-while-revalidate=86400` с ETag и ответом 304, опубликованные вакансии — `max-age=30`, всё личное — `private, no-store`. Caddy сжимает страницы и статику уровнем `zstd better`, а JSON API — уровнем zstd по умолчанию (вдвое дешевле по процессору при 1–11 % лишних байтов), отдаёт файлы `public/` с суточным кешем, а предрендеренным страницам вместо годового `s-maxage` ставит проверку ETag в браузере и минуту в общем кеше — чтобы после выкладки CDN не раздавал HTML со ссылками на старые чанки (architecture.md §12.1).

Как мерили. Нативная release-сборка API на ноутбуке Apple M5 (4 производительных и 6 энергоэффективных ядер), PostgreSQL 14 и Valkey на той же машине; база API, Postgres и генератор нагрузки делят эти 10 ядер. Нагрузка — wrk, 50 соединений, 2 с прогрева и 10 с замера. «До» — код до этой правки, «после» — с ней; стороны запускаются поочерёдно, каждая на своей копии одной и той же базы (`CREATE DATABASE … TEMPLATE`). На 240 кандидатах — медиана трёх прогонов, на 10 000 — один прогон. Генератор нагрузки не узкое место: на запросах к базе oha с 2 и 10 потоками и wrk с 2 и 4 потоками дают одно и то же; на самых дешёвых маршрутах четыре потока wrk отнимают производительные ядра у сервера, поэтому потолок ниже снят двумя. Между прогонами одного и того же кода разброс на этом стенде — до 20–30 %. Ответы до и после совпадают: 116 сравнений на каждом наборе данных — подборки с фильтрами, поиск, рекомендованные категории, профили с объяснением и полные тела сохранённой подборки, профиля и страницы поиска (числа до 0,001).

Потолок. Без базы — `/healthz` и справочник `/catalog` из памяти процесса — было ~80 000 и ~68 000 запросов в секунду, стало ~150 000 и ~140 000. Маршрут с авторизацией без запросов к базе (все категории работодателя: подпись JWT, проверка отзыва токена в Valkey, кеш) — ~41 000 → ~100 000. `/me` с двумя запросами к базе — ~12 500 → ~21 000. Процессорное время API на `/healthz` — 105 → 28 мкс на запрос. Через Docker и Caddy потолок задают они, а не API: с хоста через проброс порта Colima `/healthz` — ~17 000 запросов в секунду, изнутри виртуальной машины через Caddy — 22 000 (без JSON-журнала каждого запроса — 38 000), прямо в API — 71 000.

Почему тяжёлые запросы — тысячи, а не десятки тысяч. Страница поиска — 20 карточек кандидатов, сохранённая подборка — 30 карточек и 30 объяснений (~125 КБ JSON). Карточка — навыки, задания, бейдж прокторинга, отношение компании и живая проверка видимости; это ~0,4 мс работы производительного ядра Postgres на страницу, под нагрузкой на этом стенде — около миллисекунды процессора базы плюс ~0,5 мс API (разбор строк, JSON, системные вызовы). Поиск и подборка упираются в Postgres: он занимает ~5 из 10 ядер. От размера банка это почти не зависит — ранжирование 10 000 кандидатов в памяти занимает доли миллисекунды. Пересборку одной подборки 50 клиентов делают по очереди: пересборки одной потребности идут под блокировкой её строки.

Что показал профиль (samply, on-CPU флеймграфы под нагрузкой): - `/healthz`: две трети процессора — клонирование и удаление стека middleware маршрута и `AppState` из 12 `Arc`. axum клонирует стек маршрута на каждый запрос и ещё раз внутри каждого `from_fn`, а счётчики ссылок общие для всех ядер. - Каждый запрос к базе — два обмена: sqlx по умолчанию пингует соединение перед выдачей из пула. - Карточки — четыре запроса (карточки, отношение компании, бейджи прокторинга, живая видимость) и ещё один — компания работодателя; основной группировал широкие строки кандидатов по навыкам. - Профиль одного кандидата: Postgres заново планировал запрос карточек на каждый вызов и не переходил на общий план — 0,5 мс вместо 0,12 мс. - Проверка подписи JWT (Ed25519, ~50 мкс) — четверть процессора на маршрутах без базы. - Показ подборки: объяснения из `jsonb` переводились в текст, разбирались в `serde_json::Value`, потом в структуру и сериализовались снова — 30–40 % процессора API на этом маршруте. - Пересборка: 30 вставок по одной под блокировкой строки потребности. - Не узкие места: журнал (построчного журнала запросов нет; `info,api=debug` в JSON и `warn` дают одно и то же), метрики (~1,5 % на `/healthz`), Valkey (одна проверка отзыва токена на запрос), ранжирование и сортировка (до 12 % процессора API на поиске).

Что сделано (architecture.md §12.3): `AppState` — один `Arc`; общие middleware обёрнуты вокруг роутера один раз, а не вокруг каждого маршрута; `x-request-id` — без системного вызова. Пингуется только соединение, простоявшее больше секунды. Карточки — один запрос с подзапросами `LATERAL` (вдвое дешевле прежнего основного), видимость — в нём же; массив id передаётся так, чтобы Postgres закешировал общий план. Компания работодателя и проверенные JWT кешируются в памяти процесса без мьютексов; срок действия токена проверяется каждый раз, отзыв — как раньше, в Valkey. Объяснение в подборке хранится как `json` (миграция 0010) и отдаётся без разбора. Пересборка пишет 30 строк одним `INSERT … SELECT FROM UNNEST` и держит блокировку на три запроса вместо 32. Профиль строит объяснение по одному кандидату, не копируя пул.

Запросы в секунду и p95, мс:

Запрос	240 кандидатов: до	после	10 000: до	после
`/healthz` (без базы)	74 478 / 1,0	113 238 / 0,5	73 070 / 1,0	114 046 / 0,5
Справочник `/catalog` (из памяти)	67 501 / 1,1	110 813 / 0,5	64 309 / 1,2	103 319 / 0,6
`/me` (два запроса к базе)	11 796 / 4,6	17 059 / 3,2	12 582 / 4,2	17 001 / 3,2
Все категории работодателя	42 223 / 1,6	75 764 / 0,9	41 393 / 1,7	79 662 / 0,8
Публичный список вакансий	18 945 / 2,8	24 239 / 2,3	20 046 / 2,7	24 146 / 2,3
Рекомендованные категории потребности	11 550 / 4,7	24 253 / 2,9	10 348 / 6,2	20 626 / 2,7
Поиск по банку	2 036 / 40	2 725 / 47	2 666 / 24	3 847 / 41
Поиск по банку под потребность	2 235 / 33	2 755 / 44	2 016 / 898	2 903 / 643
Профиль с объяснением	3 157 / 21	7 704 / 8	3 032 / 25	9 746 / 5
Показ сохранённой подборки	1 777 / 36	2 508 / 54	1 194 / 52	2 124 / 44
Пересборка подборки (одна потребность)	313 / 238	503 / 149	142 / 499	272 / 282

Первые шесть строк сняты с четырьмя потоками wrk (см. выше про потолок). У поиска и подборки после правки выше и пропускная способность, и p95: запросы короче, и Postgres держит в работе все 16 соединений пула на 10 ядрах, общих с API и wrk. Хвост задаёт размер пула (`DATABASE_POOL_SIZE`): на 10 000 кандидатах при 32 соединениях пропускная способность поиска та же, что при 16, а p95 — 120 мс вместо 25; при 8 — на 9 % меньше запросов, зато p95 10 мс. Меньше потоков tokio, чем ядер, только хуже (4 потока: −12 % на поиске, −20 % на маршрутах без базы). Длинный хвост поиска под потребность на 10 000 — раз в 30 с снимок пересобирается вместе с ранжированием текста потребности (до ~170 мс), а 50 запросов ждут одну загрузку.

Docker и Caddy (240 кандидатов, wrk с хоста на :8080, один прогон «после»): поиск по банку 1 249 → 1 897 запросов в секунду, под потребность 1 224 → 1 468, профиль с объяснением 2 358 → 4 346, сохранённая подборка 830 → 1 210, пересборка 198 → 263, категории потребности 6 838 → 10 080, все категории 6 484 → 12 771, `/me` 7 777 → 9 094; `/healthz` — ~17 000 и там и там (потолок Colima). wrk не просит сжатия; браузер просит, и тогда сжатие JSON стоит Caddy заметных денег. На 8 соединениях `zstd better` тратил на ответ 0,43–0,63 мс процессора Caddy, уровень по умолчанию — 0,17–0,36 мс при ответе на 1–11 % больше (поиск 5 391 → 5 743 байта, подборка 9 237 → 10 282, вакансии 3 119 → 3 140); вакансий при этом 4 593 → 6 776 запросов в секунду. Поэтому `/api/*` сжимается уровнем по умолчанию, страницы и статика — `better`. Заодно нашлось, что при 50 одновременных ответах со сжатием Caddy с лимитом 64 МБ убивало OOM по кругу (пик — ~150 МБ): лимит поднят до 256 МБ, `GOMEMLIMIT=200MiB`.

Что ещё можно. - Карточки — главный расход базы. Их можно держать в памяти, как пул, но последняя активность, задания и бейдж прокторинга меняются без сброса поколения: пришлось бы сбрасывать кеш и при этих записях или показывать их с опозданием до 30 с. Оценка — поиск и подборка в 2–3 раза быстрее, день работы с тестами на видимость. - Окно блокировки пересборки — три обмена с базой. Конвейер запросов внутри транзакции сократил бы его до одного, но sqlx его не умеет; для разных потребностей пересборки и так идут параллельно. - JSON-журнал каждого запроса в Caddy съедает ~40 % его пропускной способности (22 000 → 38 000 `/healthz`); можно

не писать успешные чтения `/api/*` или писать выборочно — вопрос того, что нужно эксплуатации. - Проброс порта Colima ограничивает стенд ~17 000 запросами в секунду; на Linux-сервере этого слоя нет.

Индексы (миграция `0009_perf_indexes`) — только те, что оправдал `EXPLAIN (ANALYZE, BUFFERS)` на 10 000 кандидатах, 300 компаниях, 45 000 приглашениях и 90 000 просмотров профиля. Среднее время запроса (pgbench, 50 повторов):

Запрос	Было	Стало	Что изменилось
Потребность со счётчиком приглашений (каждый запрос с потребностью)	1,53 мс	0,16 мс	`invitations (need_id)`: вместо полного прохода по приглашениям
Список потребностей компании	3,27 мс	0,16 мс	тот же индекс
Ленивое истечение приглашений (перед каждым чтением приглашений)	1,09 мс	0,23 мс	`invitations (expires_at)` по живым приглашениям
Вилка категории (проверка зарплаты, публикация вакансии)	3,67 мс	0,87 мс	`user_consents (document_id, user_id)` по действующим согласиям
Спрос в категории кандидата	14,9 мс	13,6 мс	`profile_views (candidate_id)`; остальное — кеш
Загрузка пула специализации	59 мс	16 мс	навыки собираются по кандидату, а не группировкой всего соединения (24 мс), плюс индекс по согласиям

Ранжирование текста потребности по «о себе» (`ts_rank_cd`, 25 мс на специализацию) индексом не ускоряется — оно кешируется вместе со снимком, по тексту потребности. Индекс по дате создания приглашений сокращал спрос в категории ещё на 4 мс, но этот запрос теперь кешируется, и индекс не добавлен.